

Binary and Multiclass Classification Intrusion Detection System using Benchmark NSL-KDD and Machine Learning Models

1st Nilesh G. Pardeshi
MET's IOE, Nashik
Affiliated to SPPU, Pune
Nashik, India
ngpardeshi@gmail.com

2nd Dipak V. Patil
GES RHSCOEMSR, Nashik
Affiliated to SPPU, Pune
Nashik, India
dipakvpatil17@gmail.com

Abstract—In the modern world, cyber security is growing in importance. Every day, there are more and more people using the internet, and they are downloading vast amounts of data from various websites onto their gadgets. Through the introduction of various attack types, attackers aim to obtain entry to the systems of regular users. To defend regular users against intruders, a multitude of intrusion detection systems are being created. Since most of these systems were created using obsolete datasets, there is room for improvement in terms of detection rate, accuracy, and false alarm rate. The suggested system builds various shallow machine learning frameworks for binary as well as multiclass categorization. Proposed system applies shallow learning techniques like Random Forest, Decision Tree, Gaussian Naïve Bayes, Support Vector Machine, K-Nearest Neighbor, Gradient Boost, AdaBoost and Linear Discriminant Analysis on benchmark NSL-KDD dataset. Suggested system selects best features by applying SelectKBest and Correlation Features Selection methods on NSL-KDD dataset. All machine learning frameworks are build using selected features KDDTrain+ dataset. Grid-search is applied to get the optimal hyperparameters of a models which helps in the most accurate results. Testing is carried out on distinct test dataset KDDTest+. It is observed that the performance of Gradient Boosting model for binary classification is highest as compared with other models where, Accuracy- 86%, Precision- 89%, Recall- 84%, and F1 score- 85% Also, performance of Gradient Boost and Random Forest model is better for multiclass classification, in comparison with other shallow learning models where, Accuracy- 90%, Precision- 92%, Recall- 90%, and F1 score- 90%.

Keywords—*intrusion detection, shallow learning, machine learning, cyber security, NSL-KDD, KDDtrain+, kddtest+.*

I. INTRODUCTION

The majority of daily tasks, including banking, shopping, admissions, ticketing, teaching, and learning, are now done online as of COVID-19. Thus, the number of people using the internet in today's online world is growing daily. They are using their smartphones to access and transfer massive amounts of data from various places. Using various attack terminologies, malevolent individuals are attempting to gain access to the private information of regular users on the network. It is becoming more and more important to protect regular users' data from malevolent users so they can utilize various internet services without worrying about their sensitive data being misused or stolen. By spotting unusual network traffic, intrusion detection systems (IDS) safeguard the data of regular users.

Many researchers are currently developing many forms of intrusion detection systems, each with advantages and disadvantages. The majority of current systems are built with

antiquated intrusion detection datasets, such as KDDCUP 99 and DARPA 98. These datasets do not include newly defined attack definitions and do not reflect network traffic in the actual world. As a result, it is becoming increasingly important to create a successful intrusion detection system by utilizing benchmark and up-to-date intrusion detection datasets such as NSL-KDD [1]. This dataset contains newly defined attack terminologies and is similar to the majority of actual data flow across networks. The most popular method in IDS is supervised machine learning. Decision Tree [8], [9], Support Vector Machine [2], [3], [6], [7] K-Nearest Neighbors [4], [10], and Naive Bayes [5], [11]-[13] are a few examples. Most of the existing systems have not tested their work with standard test dataset. Many of them have not tuned their models, by selecting optimal hyperparameters. Furthermore, a thorough examination of models under various experimental situations is absent from the majority of currently published papers.

This research work uses various shallow machine learning models and suggested techniques to create an efficient intrusion detection system. The primary goals of this study are,

- To train and evaluate shallow machine learning models using the standard NSL-KDD dataset.
- To use SelectKBest and Correlation Feature Selection techniques to narrow down the dataset's size and choose the key attributes.
- To apply Grid-search technique for getting the optimal hyperparameters of a machine learning model, which helps in generating the most accurate results.
- To build the shallow machine learning frameworks like Gaussian Naïve Bayes, K-Nearest Neighbour, Decision Tree, Random Forest, Support Vector Machine, Gradient Boost, AdaBoost and Linear Discriminant Analysis using chosen features dataset. The test dataset is classified using the training models in binary and multiclass fashion.
- To evaluate models using test dataset such as KDD Test+ and calculate performance using various parameters like Accuracy, Precision, Recall and F1 Score.

The paper is composed with following sections. Section I covers Introduction. Section II presents Related Works in line with this study. Section III describes about NSLKDD dataset used. Section IV gives details about preprocessing techniques applied on dataset. Section V focus on proposed system, Section VI covers Machine Learning Models used. Section

VII elaborates Experimental Evaluations and Results. Section VIII gives the conclusion of this study.

II. RELATED WORK

To find significant selected input features from the NSL-KDD dataset, Dr. Saurabh Mukherjee et al. [14] employed the Characteristic Vitality Based approach for dimensionality reduction. For building the IDS system on the smaller NSL-KDD dataset, they employed the naïve-bayes algorithm. Using the NSL-KDD dataset, Kajal Rai et al. [15] created an intrusion detection framework based on the C4.5 decision tree technique and a decision tree algorithm. Information gain values for each feature are computed in order to identify the most significant attributes. The divide value is selected so that the classifier is unbiased with regard to the values that come up very often. Bhupendra Ingre et al [16] The suggested work employs Classification and Regression Tree technique with gini index as dividing standards for constructing the classifier. To choose the most significant features and minimize the dataset dimensionality, correlation-based feature selection, or CFS, is employed. The benchmark NSL-KDD dataset's independent testing data was used to evaluate the recommended system.

An intrusion detection technique depending on the Decision Tree with Elimination of Recursive Features in collaborative learning is proposed in this research by Wenjuan Lian et al. [17]. Ultimately, a set of cross-validation comparative tests conducted on the KDD CUP 99 and NSL-KDD datasets show that the Decision Tree with Iterative Feature Elimination and stacking-based method may enhance the IDS's performance more effectively. Farnaaz Nabila et al. [18] The four forms of attacks that are detected by the Random Forest method are DOS, probing, U2R, and R2L. For categorization, a 10-fold cross validation was used. In order to get over the problems with information gain, feature selection is done to the dataset utilizing symmetrical unpredictability of attributes. To balance the dataset, Weijinxia, Longchun, et al. [19] enhanced the synthetic minority oversampling method. The classifier for multiple assault type identification was trained using the random forest approach after characteristics were reduced using random forest and correlation analysis.

To lower the dimensionality of the dataset, Abebe Tesfahun [20] presented the Random Forests classifier using synthetic minority oversampling technique and information gain-based feature choice. The NSL-KDD dataset, which is balanced and of smaller size, is used to train Random Forest. For the purpose of training the NSL-KDD dataset, B. Basaveswara Rao [21] suggested the Indexed Partial Distance Search utilizing the k-Nearest Neighbour method. They tested the model's functionality using the 10-fold cross validation approach. The Support Vector Machine (SVM) was proposed by Md. Al Mehedi Hasan [22] using various kernels. The outcomes of the investigations shows that the RBF kernel outperforms the linear and polynomial kernels in terms of detection rate. Luis Alfredo [23] trains and tests a Support Vector Machine model using every feature of the NSL-KDD dataset. After that, this framework will be using a feature choice technique to take out the most relevant characteristics originating with the NSL-KDD dataset. The framework is reconstructed using a subset of the attribute's dataset.

Emmanuel Mugabo [24] Information Gain is employed to identify the pertinent characteristics and eliminate superfluous characteristics since KDD datasets contain superfluous and

unnecessary features. SVM classifier is used to categorize networking information into typical and attack behaviours. Using the most recently dataset, CICIDS-2017, Arif Yulianto [25] et al. built an intrusion detection framework with enhanced functionality. The synthetic minority oversampling technique is applied to deal with the unbalanced training dataset. Furthermore, for the crucial characteristic's selection, Principal Component Analysis and Ensemble Feature Selection are employed. Relying on tree-structured machine learning frameworks and the CICIDS-2017 dataset, Li Yang et al. suggested an intrusion detection framework. The enhanced detecting efficiency of the outlined system is made possible by the suggested feature choice and ensemble learning techniques. Two Long Short-Term Memory relied Intrusion Detection Frameworks with retrieval of profound characteristic were suggested by Anbang Wang [27]: the IDS with multiple classes feature extraction and the two-class feature retrieval.

Ziadoon Kamil Maseer et al. [26] used the CICIDS-2017 dataset to build various models using well-known supervised and unsupervised machine learning algorithms. Additionally, productive and successful machine learning-based Advanced Intrusion Detection System for computers and networks was determined. Weiming Hu et al. [28] proposed an intrusion detection algorithm based on the AdaBoost algorithm. In the algorithm, decision stumps are utilized as weak classifiers. The decision rules are provided for both categorical and continuous features. Adjustable starting weights and a straightforward approach for avoiding overfitting are adopted to enhance the efficiency of the algorithm.

III. NSLKDD DATASET

The KDDCUP 99 dataset served as its source. The NSL-KDD dataset partially addresses the shortcomings of KDDCUP 99, including unnecessary and duplicate data as well as the unbalanced nature of data in various attack categories. DoS, Probe, U2R, and R2L kinds of attacks are among the four main attack categories that NSL-KDD comprises, as shown in Table 1. There are 41 features in all, divided into four categories: Intrinsic features collected from packed header, Content features include actual packet information, Time based features are generated from information of traffic connection intervals of 2 seconds and Host based features accounts all series of connections. Three features flag, protocol_type, and service are categorical, and 39 of the features are numerical. The majority of researchers have trained and tested their intrusion detection systems using this dataset. Several machine learning models have been trained using KDDTrain+, which has 1,25,973 record samples, and tested using KDDTest+, which has 22544 record samples. Since the train and test datasets differ, actual testing is done for several models on unknown data.

TABLE I. ATTACK TYPES IN THE NSL-KDD DATASET

Types of Attacks	Attack Definition
Denial of Service (DoS)	To overload connections so that the service is unavailable
Probe	To obtain crucial information (port scanning)
Remote to Local (R2L)	To allow remote machines to access local resources
User to Root (U2R)	To obtain Super User rights

IV. DATASET PREPROCESSING

Proposed system first preprocesses the data by assigning numerical label tags [0, 1, 2, 3, 4] rather than [Normal, DoS, Probe, R2L, U2R] appropriately, in order to conduct data investigation utilizing training sets.

A. Feature Scaling

Due to the fact that the dataset contains values for some input attributes with varying scales, feature scaling is necessary. The Standard Scalar function in the sklearn library is used to standardize these attributes. Every feature in the dataset will be independently normalized by Standard Scalar, resulting in a standard deviation(σ) = 1 and a mean(μ) = 0 for each feature. Standard scaling will be computed as follows:

$$Z = \frac{X - \mu}{\sigma} \quad (1)$$

$$\mu = \frac{1}{N} \sum_{i=1}^N X_i \quad (2)$$

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (X_i - \mu)^2} \quad (3)$$

where Z = value of feature after scaling, as in (1), X = initial value of a feature, μ = average of every value of a feature, as in (2), and σ = feature's standard deviation, as in (3).

B. get_dummies

The categorical variables can be changed into dummy variables in Pandas using the `get_dummies()` function. The availability of every category in the initial data is indicated by a binary value (1 or 0) in a new column that represents the category. In NSL-KDD dataset there are three categorical features. `get_dummies()` function will introduce new columns corresponding to each category in these features. `flag` with 11 categories represented by 11 different columns. `protocol_type` with 3 categories represented by 3 different columns and `service` with 70 categories represented by 70 different columns. Total 84 new columns are getting introduced in NSL_KDD dataset.

C. SelectKBest

SelectKBest method is used for scoring and ranking the features according to how they relate to the outcome variable using statistical tests such as the ANOVA F-test, mutual information score and chi-squared test. The K features that have the best scores are then chosen to be a part of the final feature subset. The parameters of SelectKBest method are k and the score function.

1) *Score function*: `mutual_info_classif` is applied to assess the significance of the attribute. Its foundation is the idea of mutual information, which quantifies how much information is shared between two variables. The mutual information between each characteristic and the target variable is calculated. High ratings will be awarded to features that are very instructive about the intended variable.

2) K : is the number that denotes the highest number of attributes that should be chosen.

D. Correlation Feature Selection (CFS)

The CFS algorithm operates by initially calculating the correlation among every characteristic as well as the intended

variable. Then, it computes the correlation among every pair of features. It then chooses the subset of attributes having the lowest correlation among themselves and the highest correlation with the target variable. The measure of Symmetrical Uncertainty, which considers the mutual information between the characteristics and the intended variable, is used to determine the correlation for classification tasks.

V. RECOMMENDED FRAMEWORK

The suggested framework in Fig. 1 produces a functional Intrusion Detection System through the use of various machine learning models like Random Forest, Decision Tree, Gaussian Naïve Bayes, Support Vector Machine, K-Nearest Neighbor, Gradient Boost, AdaBoost and Linear Discriminant Analysis on standard NSL-KDD dataset. The initial dataset is pre-processed before various machine learning methods are applied. All categorical feature values are transformed into binary values with the help of the `get_dummies()` method in Pandas. Each category is transformed into a new column with binary value (1 or 0) indicating the presence of the category in the original data.

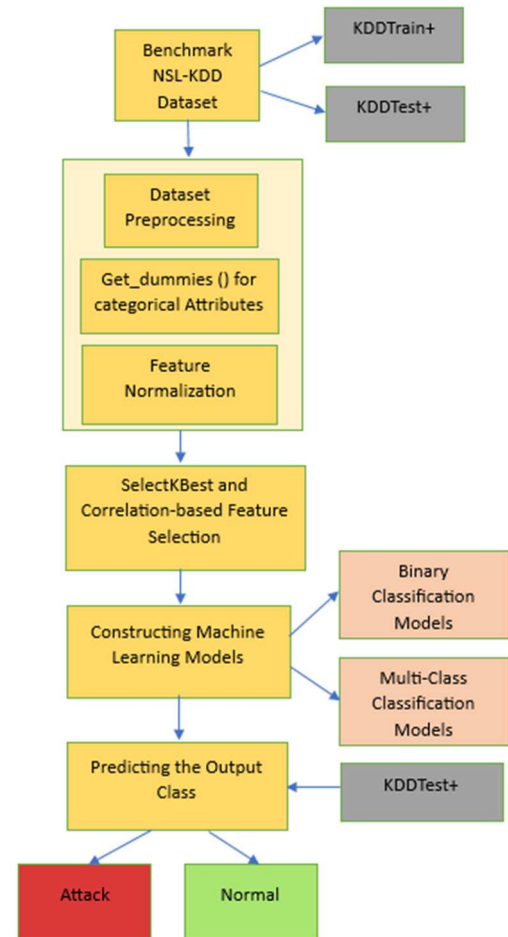


Fig. 1. Proposed System Architecture.

All the attributes are normalized using standard scalar, resulting in mean(μ) = 0 and standard deviation(σ) = 1 for every attribute. By choosing the most significant characteristics from the dataset, the dimensionality of the pre-processed datasets is decreased. SelectKBest and Correlation Feature selection methods are used for selecting the significant attributes from the datasets. Selected attributes

smaller dimensions dataset is utilized for the construction of various machine learning frameworks. Grid-search technique is applied for getting the optimal hyperparameters of a machine learning model, which helps in generating the most accurate results. The functionality of the constructed frameworks is assessed by testing them on distinct test dataset.

VI. MACHINE LEARNING MODELS

A. Naïve Bayes

It is probabilistic machine learning algorithm based on Bayes Theorem. It assumes that features that go into the model is independent of each other. The Bayes Rule is a method for determining $P(Y|X)$, where X =attributes and Y =response, from $P(X|Y)$, which is identified from the training dataset.

For records within the test dataset, the X would be known as though Y is unknown. In this problem, there are multiple X variables. As features are independent, the Bayes Rule can be extended to what is called Naïve Bayes, as in (4).

$$P(Y = k|X_1 \dots X_n) = \frac{P(X_1|Y=k) \dots P(X_n|Y=k) * P(Y=k)}{P(X_1) * P(X_2) \dots P(X_n)} \quad (4)$$

Best hyperparameter combination is: 'priors': None, 'var_smoothing': 1e-05

B. Decision Tree

S is assumed as the tree's root node, holding the whole dataset. Choose the most favorable feature from the dataset by utilizing the Attribute Selection Measure: Gini Index, as in (5).

$$Gini = 1 - \sum_{i=1}^n (p_i)^2 \quad (5)$$

Whereas n is quantity of classes and p_i is probability of belonging to class i . Divide ' S ' into subgroups that incorporate possible values for the optimal characteristics. Generate the decision tree node, which comprises of the top attribute. Iteratively create novel decision trees by utilizing the subgroups of the dataset generated. Follow this route until it is not possible for you to categorize the nodes anymore; then designate the last node as a leaf node. Best hyperparameters selected as: 'max_depth': 10, and 'min_samples_split': 2

C. Random Forest

Random Forest is a Supervised machine learning algorithm depending on collaborative learning. By merging several decision trees, the random forest technique builds a forest of the trees. Choose N instances randomly from the dataset. Using these N instances, build a decision tree. Choose quantity of trees as per requirement of algorithm and iterate through above steps. In case of categorization problem, every tree within the forest forecasts the category that the newly created instance falls within. The category that receives highest votes is finally assign the new instance. Best hyperparameters selected as: 'max_depth': None, 'max_features': 'sqrt', and 'n_estimators': 1000

D. K-Nearest Neighbor

It is an easiest supervised machine learning algorithm. It assumes the similarity between new case and available cases and put the new case into category that is most similar to available categories. Compute the distance among the data instance and each of the other instance with the aid of

Euclidean distance technique. Euclidean distance among A (X_1, Y_1) and B (X_2, Y_2) is computed as in (6).

$$\sqrt{(X_2 - X_1)^2 + (Y_2 - Y_1)^2} \quad (6)$$

Sort the distances values in increasing order. Choose the highest k values among the distances sorted. Among these k neighbors, count the number of the data points in each category. Assign the new data point to that category for which the number of the neighbor is maximum. Best hyperparameters selected as: 'n_neighbors': 1, and 'weights': 'uniform'

E. Support Vector Machine

Support Vector Machine (SVM) model will create the best line or decision boundary that can segregate n -dimensional space into classes so that the new data point can be easily put in the correct category in the future. Best decision boundary is called a hyperplane. SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called as support vectors. The distance between the vectors and the hyperplane is called as margin. And the goal of SVM is to maximize this margin. The hyperplane with maximum margin is called the optimal hyperplane. Best hyperparameters selected as: 'C': 5, and 'gamma': 0.1

F. Linear Discriminant Analysis

Linear Discriminant Analysis (LDA), make easier the modelling of differentiations among groups, productively segregating two or more classes. LDA works by projecting attributes from a higher-dimensional space into a lower-dimensional one. In machine learning, LDA performs as a supervised learning model particularly build for classification jobs. Its objective is to find out a linear combination of attributes that perfectly separate classes within a dataset. Best hyperparameters selected as 'shrinkage': None and 'solver': lsqr.

G. Gradient Boost

Gradient boosting is a machine learning ensemble method which integrates the outputs of more than one weak learners, generally decision trees, in a sequential manner. Its objective is to enhance overall predictive performance by upgrading the model's weights depending on the errors of earlier iterations, progressively lowering prediction errors and improving the model's accuracy. It constructs models in a sequential manner and these subsequent models try to lower the errors of the earlier model. This is done by constructing a new model $F_m(x)$ as given in (7) on the errors or residuals of the earlier model $F_{m-1}(x)$. Where γ_m is a step length and $h_m(x)$ is a decision tree.

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x) \quad (7)$$

Best hyperparameters selected as 'learning_rate': 0.2, 'max_depth': 4, and 'n_estimators': 200

H. AdaBoost

AdaBoost algorithm, abbreviation for Adaptive Boosting, is a Enhancing strategy used as an Ensemble approach in Machine Learning. It is called Adaptive Boosting as the weights are re-allocated to each sample, with larger weights assigned to wrongly categorized samples. It constructs a model and assigns similar weights to all the data points. It then allocates larger weights to data points that are incorrectly

categorized. Now all the data points with larger weights are given more weightage in the upcoming model. It will continue training models until and unless a lower error is obtained. Best hyperparameters selected as: 'algorithm': 'SAMME.R', 'learning_rate': 1, and 'n_estimators': 200

VII. EXPERIMENTAL EVALUATION AND RESULT

On an Intel(R) Core (TM) i5 CPU running Ubuntu 20.04 LTS with 8GB RAM, experiments are conducted. Jupyter Notebook and Python are utilized for experiments with the most recent version of the Anaconda Framework. Experiments are carried out with the help of different shallow machine learning models such as Random Forest, Decision Tree, Gaussian Naïve Bayes, Support Vector Machine, K-Nearest Neighbor, Gradient Boost, AdaBoost, and Linear Discriminant Analysis. These models are constructed on standard NSL-KDD dataset with picked out features. Selection of features is done by SelectKBest and Correlation Feature Selection methods. KDDTrain+ which has 1,25,973 record samples is utilized for training purpose, while KDDTest+ which has 22544 record samples is utilized for evaluation.

A. Performance Measures

Five measures are used in this paper i.e., 1) Detection Rate (Recall), 2) False Alarm Rate, 3) Accuracy, 4) Precision, and 5) F1 Score. The measures were computed using the following four dimensions, which are listed below:

- TP: represents all those test cases which are attacks and also predicted as an attacks
- TN: represents all those test cases which are normal and predicted also as a normal
- FP: represents all those test cases which are normal but predicted as an attacks
- FN: represents all those test cases which are attacks but predicted as a normal

1) *True Positive Rate (TPR) | Sensitivity | Recall | Detection Rate (DR)*: It is the proportion of accurately predicted attacks to all attacks. It illustrates how well the model can identify the attacks, as in (8).

$$TPR = \frac{TP}{TP+FN} \quad (8)$$

2) *False Positive Rate (FPR) | False Alarm Rate*: It's the proportion of regular occurrences that are mistakenly labeled as attacks to the overall number of normal occurrences, as in (9).

$$FPR = \frac{FP}{FP+TN} \quad (9)$$

3) *Accuracy | Classification Rate*: A measurement of the IDS's precision in identifying typical or unusual traffic patterns. It is the proportion of accurately categorized samples to all samples, as in (10).

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} \quad (10)$$

4) *Precision (P)*: It is proportion of actual positive samples to those that were anticipated to be positive. It stands for the assurance that an attack has been detected, as in (11).

$$Precision = \frac{TP}{TP+FP} \quad (11)$$

5) *F1 Score (F)*: It is described as the symmetric average of the recall and the precision, as in (12).

$$F1 \text{ Score} = \frac{2TP}{2TP+FP+FN} = \frac{2 \cdot P \cdot R}{P+R} \quad (12)$$

B. Confusion Matrix for IDS

After evaluating the model and projecting the test set's outcomes, the confusion matrix for each experiment is shown. Confusion matrix shows four potential outcomes such as TP, TN, FP, FN as shown in Table 2.

TABLE II. CONFUSION MATRIX

Confusion Matrix		Predicted Class	
		Yes (Attack)	No (Normal)
Actual Class	Yes (Attack)	TP	FN
	No (Normal)	FP	TN

Table 3 represents performance of different models for binary categorization, while utilizing KDDTrain+ as training dataset and KDDTest+ as an evaluating dataset. It is observed that Gradient Boosting model is giving highest performance as compare to other models. where, Accuracy by using Grid Search Cross-Validation - 99.88%, Accuracy- 86%, Precision- 89%, Recall- 84%, and F1 score- 85%. Confusion Matrix of highest performing Gradient Boost model is represented by Fig. 2.

TABLE III. PERFORMANCE OF BINARY CLASSIFICATION

ML Models	Accuracy using GridSearchCV	Accuracy	Precision	Recall	F1 Score
Gaussian NB	95.64	57	66	50	36
SVM	99.61	79	82	81	79
Decision Tree	99.70	82	83	81	81
RF	99.89	76	78	78	76
KNN	99.71	77	81	79	77
AdaBoost	99.45	77	79	79	77
LDA	96.41	77	81	79	76
Gradient Boost	99.88	86	89	84	85

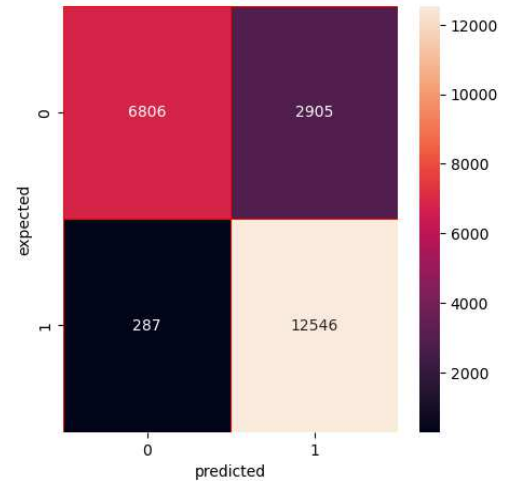


Fig. 2. Confusion Matrix of Binary Classification Gradient Boost Model.

Table 4 represents performance of different models for multiclass classification, while using KDDTrain+ as training dataset and KDDTest+ as a testing dataset. It is observed that Random Forest and Gradient Boosting models are giving highest performance as compare to other models. Confusion matrix of highest performing multiclass classification Random Forest and Gradient Boost model are shown in Fig. 3 and Fig. 4 respectively

TABLE IV. PERFORMANCE OF MULTICLASS CLASSIFICATION

ML Models	Accuracy	Precision	Recall	F1 Score
Gaussian NB	80	89	80	82
SVM	88	89	88	89
Decision Tree	89	93	89	90
RF	90	92	90	90
KNN	87	89	87	87
LDA	84	88	84	85
Gradient Boost	90	91	90	90
AdaBoost	70	84	70	70

Table 5 shows the comparative results of proposed study with existing study [29]. It is found that proposed work gives improved results in terms of accuracy, precision, recall and F1 score. As proposed system uses SelectKBest and Correlation Feature Selection methods for most relevant feature subset selection. Also Grid-search technique is used for selecting optimal hyperparameters of various machine

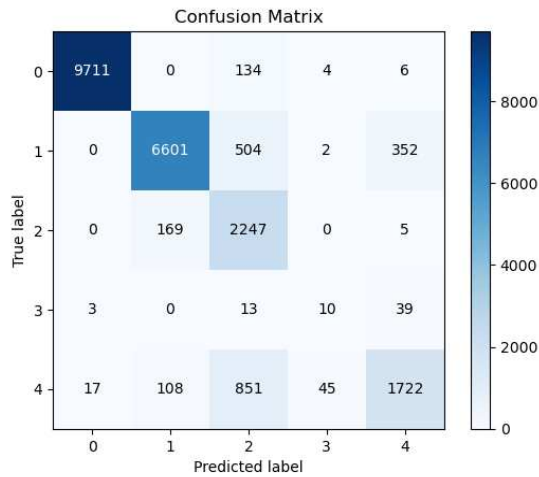


Fig. 3. Confusion Matrix of Multiclass Classification Random Forest Model

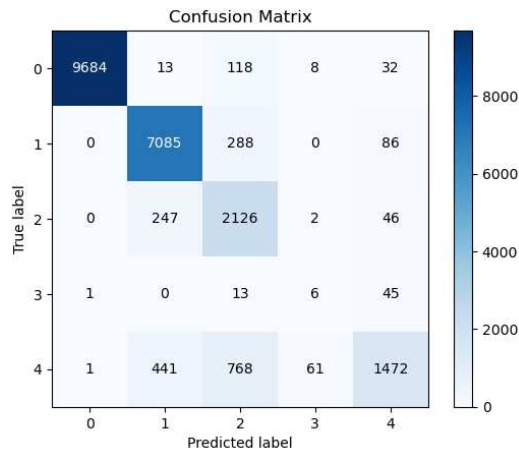


Fig. 4. Confusion Matrix of Multiclass Classification Gradient Boost Model

learning models. Therefore, constructed models are predicting most accurate and enhanced results as compared to earlier works.

TABLE V. COMPARISON OF MULTICLASS CLASSIFICATION WITH [29]

ML Models	Accuracy		Precision		Recall		F1 Score	
	Proposed	[29]	Proposed	[29]	Proposed	[29]	Proposed	[29]
GNB	80	36.8	89	39	80	36.8	82	27.7
SVM	88	79.4	89	73.2	88	79.4	89	74.3
DT	89	75	93	76.7	89	75	90	70.2
RF	90	74.8	92	74.5	90	74.8	90	69.9
KNN	87	73.4	89	72.5	87	73.4	87	68.4
LDA	84	-	88	-	84	-	85	-
GBoost	90	74.6	91	79.3	90	74.6	90	70.1
ABoost	70	-	84	-	70	-	70	-

VIII. CONCLUSION

Proposed system represents an efficient intrusion detection system in the present era of cyber security. Various preprocessing methods are used to enhance the performance of the classifiers. Models are trained using selected subset of features from KDDTrain+. SelectKBest and Correlation Feature Selection methods are used for feature subset selection. Grid-search is used for selecting optimal hyperparameters of various machine learning models. Various shallow machine learning models like AdaBoost, Gradient Boost, Random Forest, Decision Tree, K-Nearest Neighbour, Support Vector Machine, Gaussian Naïve Bayes, and Linear Discriminant Analysis are constructed with selected hyperparameters.

Benchmark NSL-KDD dataset is used for training and evaluation of the machine learning frameworks. Distinct models are constructed for binary and multiclass classification using Benchmark dataset. KDDTrain+ dataset is used for training the model and KDDTest+ is used for model testing. Experimental analysis shows that performance of Gradient Boost model is better for binary classification, in comparison with other shallow learning models where, Accuracy by using GridSearchCV- 99.88%, Accuracy- 86%, Precision- 89%, Recall- 84%, and F1 score- 85%. Also, performance of Gradient Boost and Random Forest model is better for multiclass classification, in comparison with other shallow learning models where, Accuracy- 90%, Precision- 92%, Recall-90%, and F1 score- 90%.

There are further improvement opportunities, to select most appropriate features by trying other deep feature selection methods. Also, deep learning techniques can be used for constructing machine learning models.

ACKNOWLEDGMENT

I am very much thankful to my research guide Dr. Dipak V. Patil for his valuable guidance and constant motivation during this research work and also for writing this paper. Also, I thanks to my research centre "MET's Institute of Engineering, Nashik" including Heads, Research coordinator for providing valuable suggestions and appropriate direction for the research work.

REFERENCES

- [1] Tavallae M., Bagheri E., Lu W., and Ghorbani A. A., "A detailed analysis of the KDD CUP 99 data set", In Proc. IEEE Symp. Comput. Intell. Secur. Defense Appl., pp. 1-6, 2009.
- [2] Ahmad I., Basher M., Iqbal M. J., and Raheem A., "Performance comparison of support vector machine, random forest, and extreme

- learning machine for intrusion detection", IEEE Access 6: 33789-33795, 2018.
- [3] Mill J. and Inoue A., "Support vector classifiers and network intrusion detection", In Proc. IEEE Int. Conf. Fuzzy Syst., pp. 407-410, 2004.
 - [4] Li W., Yi P., Wu Y., Pan L., and Li J., "A new intrusion detection system based on KNN classification algorithm in wireless sensor network", J. Electr. Comput. Eng. pp. 1-8, 2014.
 - [5] Zhang B., Liu Z., Jia Y., Ren J., and Zhao X., "Network intrusion detection method based on PCA and Bayes algorithm", Secur. Commun. Netw. pp. 1-11, 2018.
 - [6] Tao P., Sun Z., and Sun Z., "An improved intrusion detection algorithm based on GA and SVM", IEEE Access 6: 13624-13631, 2018.
 - [7] Zhang Y., Yang Q., Lambbotharan S., Kyriakopoulos K., Ghafir I., and AsSadhan B., "Anomaly-based network intrusion detection using SVM", In Proc. 11th Int. Conf. Wireless Commun. Signal Process. (WCSP), pp. 1-6, 2019.
 - [8] Li M., "Application of CART decision tree combined with PCA algorithm in intrusion detection", In Proc. 8th IEEE Int. Conf. Softw. Eng. Service Sci. (ICSESS), pp. 38-41, 2017.
 - [9] Sahu S. and Mehtre B. M., "Network intrusion detection system using J48 decision tree", In Proc. Int. Conf. Adv. Comput., Commun. Informat. (ICACCI), pp. 2023-2026, 2015.
 - [10] Liao Y. and Vemuri V., "Use of K-nearest neighbor classifier for intrusion detection", Comput. Secur., 21(5): 439-448, 2002.
 - [11] Panda M. and Patra M. R., "Network intrusion detection using Naïve Bayes", Int. J. Comput. Sci. Netw. Secur. 7(12): 258-263, 2007.
 - [12] Fouladi R. F., Kayatas C. E., and Anarim E., "Frequency based DDoS attack detection approach using naive Bayes classification", In Proc. 39th Int. Conf. Telecommun. Signal Process. (TSP), pp. 104-107, 2016.
 - [13] Mukherjee S. and Sharma N., "Intrusion detection using naive Bayes classifier with feature reduction", Proc. Technol. 4: 119-128, 2012.
 - [14] Dr. Saurabh Mukherjee, Neelam Sharma, "Intrusion Detection using Naive Bayes Classifier with Feature Reduction", Procedia Technology vol. 4, pp. 119 – 128, 2012.
 - [15] Kajal Rai, M. Syamala Devi, Ajay Guleria, "Decision Tree Based Algorithm for Intrusion Detection" Int. J. Advanced Networking and Applications, vol. 7, pp. 2828-2834, 2016.
 - [16] Bhupendra Ingre, Anamika Yadav, and Atul Kumar Soni, "Decision Tree Based Intrusion Detection System for NSL-KDD Dataset", Information and Communication, Technology for Intelligent Systems vol. 2, Springer International Publishing AG, 2018.
 - [17] Wenjuan Lian, Guoqing Nie, Bin Jia, Dandan Shi, Qi Fan, and Yongquan Liang, "An Intrusion Detection Method Based on Decision Tree-Recursive Feature Elimination in Ensemble Learning", Hindawi Mathematical Problems in Engineering, pp. 15, 2020.
 - [18] Nabila Farnaaz and M. A. Jabbar, Twelveth International Multi-Conference on Information Processing-2016 (IMCIP-2016), "Random Forest Modeling for Network Intrusion Detection System", Procedia Computer Science vol. 89, pp. 213 – 217, 2016.
 - [19] Weijinxia, Longchun, Wanwei, Zhaojing, Duguanyao and Yangfan, "An Effective Intrusion Detection Model based on Random Forest Algorithm with I-SMOTE", In Proceedings of the 23rd International Conference on Enterprise Information Systems vol. 1, pp. 175-182, 2021.
 - [20] B. Basaveswara Rao and K. Swathi, "Fast kNN Classifiers for Network Intrusion Detection System", Indian Journal of Science and Technology, vol. 10, 2017.
 - [21] Md. Al Mehedi Hasan, Mohammed Nasser, Biprodip Pal, "On the KDD'99 Dataset: Support Vector Machine Based Intrusion Detection System (IDS) with Different Kernels", International Journal of Electronics Communication and Computer Engineering, vol. 4, 2013.
 - [22] Md. Al Mehedi Hasan, Shuxiang Xu, Mir Md. Jahangir Kabir and Shamim Ahmad, "Performance Evaluation of Different Kernels for Support Vector Machine Used in Intrusion Detection System" International Journal of Computer Networks & Communications (IJCNC), vol. 8, 2016.
 - [23] Luis Alfredo A'lvarez Almeida, Juan Carlos Martinez Santos, "Evaluating Features Selection on NSL-KDD Data-Set to Train a Support Vector Machine-Based Intrusion Detection System", 2019.
 - [24] Arif Yulianto, Parman Sukarno and Novian Anggis Suwastika, "Improving AdaBoost-based Intrusion Detection System (IDS) Performance on CIC IDS 2017 Dataset". The 2nd International Conference on Data and Information Science, IOP Conf. Series: Journal of Physics: Conf. Series 1192, 2019.
 - [25] Li Yang, Abdallah Moubayed, Ismail Hamieh, Abdallah Shami, "Tree-based Intelligent Intrusion Detection System in Internet of Vehicles", IEEE Conference, 2019.
 - [26] Ziadoon Kamil Maseer, Robiah Yusof, Nazrulazhar Bahaman, Salama A. Mostafa, and Cik Feresah Mohd Foozy, "Benchmarking of Machine Learning for Anomaly Based Intrusion Detection Systems in the CICIDS2017 Dataset", IEEE Access, 2021.
 - [27] Anbang Wang, Xinyu Gong, and Jialiang Lu, "Deep Feature Extraction in Intrusion Detection System", 2019 IEEE International Conference on Smart Cloud (SmartCloud), 2019.
 - [28] Weiming Hu, Wei Hu, Steve Maybank, "AdaBoost-Based Algorithm for Network Intrusion Detection", IEEE Transactions On Systems, Man, And Cybernetics—Part B: Cybernetics, vol. 38, April 2008.
 - [29] Pardeshi N. G., Patil D. V., "Applying Gini Importance and RFE Methods for Feature Selection in Shallow Learning Models for Implementing Effective Intrusion Detection System", In Proc. of Int. Conf. on Applications of Machine Intelligence and Data Analytics (ICAMIDA), pp. 214-234, 2023.